04

JavaScript kompakt

Sprache · DOM · Events · Event Loop · Fetch · Ausblick TypeScript
Web-Applikationen · Bachelor Informatik · HTWG Konstanz · WS 2026/27 ·
Haverbeke, Eloquent JavaScript · MDN: JavaScript Guide

Lernziele

- ✓ Ein Skript als Modul in eine Seite einbinden und mit der Browser-Konsole arbeiten.
- ✓ Die wichtigsten Unterschiede zwischen JavaScript und Java benennen: dynamische Typen, `let` und `const`, `===`, Funktionen als Werte.
- ✓ Mit Objekten, Arrays, Destructuring, Arrow Functions und Template Literals umgehen.
- ✓ Elemente im DOM finden, verändern und neu erzeugen, und begründen, warum `textContent` sicherer ist als `innerHTML`.
- ✓ Auf Events reagieren und mit `preventDefault` das Standardverhalten des Browsers abschalten.
- ✓ Erklären, wie der Event Loop mit Call Stack, Microtasks und Tasks arbeitet.
- ✓ Daten mit `fetch`, `async` und `await` laden und Fehler einschließlich HTTP-Statuscodes behandeln.

Worum es heute geht

Die Hackathon-Seite aus VL 2 sieht inzwischen ordentlich aus, kann aber nichts. Drei Dinge stören:

- Das Programm steht fest im HTML. Ändert sich ein Termin, muss jemand das HTML anfassen. Die Daten gehören eigentlich in eine eigene Datei und später in eine API.
- Das Anmeldeformular schickt beim Klick auf „Anmelden“ einen `POST` an `/anmeldung`, eine Adresse, die es nicht gibt. Die Seite ist danach weg, und eine Rückmeldung gibt es nicht. Wie eine solche Anfrage aussieht, kennt ihr aus VL 3.
- Das Formular prüft nichts. Ob jemand eine HTWG-Adresse angibt, merkt erst, wer die Anmeldungen liest.

In diesem Kapitel bauen wir **Stufe 4** der Seite: dasselbe HTML und CSS wie in VL 2, dazu eine Datei `app.js`. Am Ende kommt das Programm aus einer Datei `programm.json`, und das Formular prüft die Eingaben selbst. Jedes Werkzeug, das wir dafür brauchen, kommt in genau diesem Kapitel vor. Das vollständige Beispiel liegt in Moodle.

★ HTML beschreibt die Struktur, CSS die Gestaltung und JavaScript das Verhalten. JavaScript ist die einzige Programmiersprache, die jeder Browser direkt ausführt. Alles, was später React macht, beruht auf den Mitteln aus diesem Kapitel.

JavaScript einbinden

Ein Skript wird mit einem `script`-Element eingebunden. Wir schreiben es in den `head`:

```
<head>
  <link rel="stylesheet" href="styles.css">
  <script type="module" src="app.js"></script>
</head>
```

Das Attribut `type="module"` hat zwei Wirkungen. Erstens wartet der Browser mit dem Ausführen, bis das ganze HTML gelesen ist. Das Skript findet also alle Elemente der Seite, obwohl es im `head` steht. Zweitens darf das Skript andere Dateien mit `import` laden. Das brauchen wir spätestens mit React in VL 5.

Euer wichtigstes Werkzeug ist ab jetzt die **Konsole** der Entwicklerwerkzeuge (`F12`). Dort erscheinen Fehlermeldungen, Ausgaben von `console.log` landen dort, und ihr könnt JavaScript direkt eintippen. Probiert auf der Hackathon-Seite:

```
document.querySelector("h1").textContent = "Hallo VL 4";
```

Die Überschrift ändert sich sofort, ohne dass die Seite neu geladen wird. Dieser Satz enthält im Kleinen das ganze Kapitel: ein Element finden und verändern.

⚠ **Achtung:** Ab heute funktioniert der Doppelklick auf `index.html` nicht mehr. Ein Modul und `fetch` laufen nur, wenn die Seite über HTTP kommt. Öffnet ihr sie als Datei, bleibt die Seite bei „Programm wird geladen …“ hängen, und in der Konsole steht ein Fehler mit dem Wort CORS. Startet stattdessen im Projektordner einen kleinen Server, zum Beispiel mit `python -m http.server 8000`, und öffnet `http://localhost:8000`. Was CORS genau ist, behandeln wir in VL 12.

Die Sprache für Java-Kenner

JavaScript sieht auf den ersten Blick aus wie Java: geschweifte Klammern, `if`, `for`, Semikolons. Es tickt aber an einigen Stellen grundlegend anders.

Typen hängen am Wert

In Java hat eine Variable einen festen Typ. In JavaScript hat nur der Wert einen Typ, die Variable kann nacheinander Verschiedenes enthalten:

```
let x = 42;
x = "zweiundvierzig";   // erlaubt
```

Es gibt keinen Compiler, der vorher prüft, ob die Typen zusammenpassen. Fehler zeigen sich erst zur Laufzeit. Genau diese Lücke schließt TypeScript, dazu am Ende des Kapitels mehr.

let und const statt var

Variablen deklariert ihr mit `const`, wenn sie nicht neu zugewiesen werden, sonst mit `let`. Das alte `var` gilt nicht für einen Block, sondern für die ganze Funktion, und führt zu schwer auffindbaren Fehlern. Ihr trefft es nur noch in altem Code.

Wichtig: `const` schützt die Bindung, nicht den Inhalt. Ein mit `const` deklariertes Array könnt ihr weiterhin mit `push` verändern, ihr könnt der Variablen nur kein neues Array zuweisen.

Immer ===

JavaScript wandelt Typen beim Vergleich mit `==` stillschweigend um. Deshalb ist `"5" == 5` wahr und sogar `[] == false`. Der strikte Vergleich `===` vergleicht Wert **und** Typ, ohne umzuwandeln. Verwendet immer `===` und `!==`.

Funktionen sind Werte

Eine Funktion ist in JavaScript ein Wert wie eine Zahl oder ein String. Ihr könnt sie in einer Variablen speichern und an eine andere Funktion übergeben. Die kurze Schreibweise dafür heißt **Arrow Function**:

```
const verdoppeln = (n) => n * 2;
[1, 2, 3].map(verdoppeln);   // [2, 4, 6]
```

Auf dieser Idee beruhen Event-Listener, `map` und `filter`, Promises und später fast jede Zeile React.

Objekte, Arrays und moderne Schreibweisen

Daten werden in JavaScript fast immer als Objekte und Arrays dargestellt. Ein Programmpunkt des Hackathons sieht so aus:

```
const punkt = { zeit: "09:00", titel: "Kickoff",
                text: "Teams finden, Aufgaben verteilen" };
```

Vier Schreibweisen begegnen euch ständig:

- **Destructuring** holt Felder heraus: `const { zeit, titel } = punkt;`
- **Template Literals** setzen Werte in Text ein: `` `${titel} beginnt um ${zeit}` ``
- **Spread** kopiert und erweitert: `const neu = { ...punkt, raum: "F-033" };`
- `map` und `filter` erzeugen aus einem Array ein neues, ohne Schleife: `programm.map((p) => p.titel)` liefert alle Titel.

> 💡 **Tipp:** `map` ist in React der Normalfall: Aus einer Liste von Daten wird eine Liste von Elementen. Wer `map` heute sicher beherrscht, hat es nächste Woche leichter.

Das DOM

Der Browser liest das HTML genau einmal und baut daraus einen Baum von Objekten, das **Document Object Model** (DOM). Danach zählt nur noch dieser Baum. Ändert JavaScript den Baum, zeichnet der Browser die Seite neu.

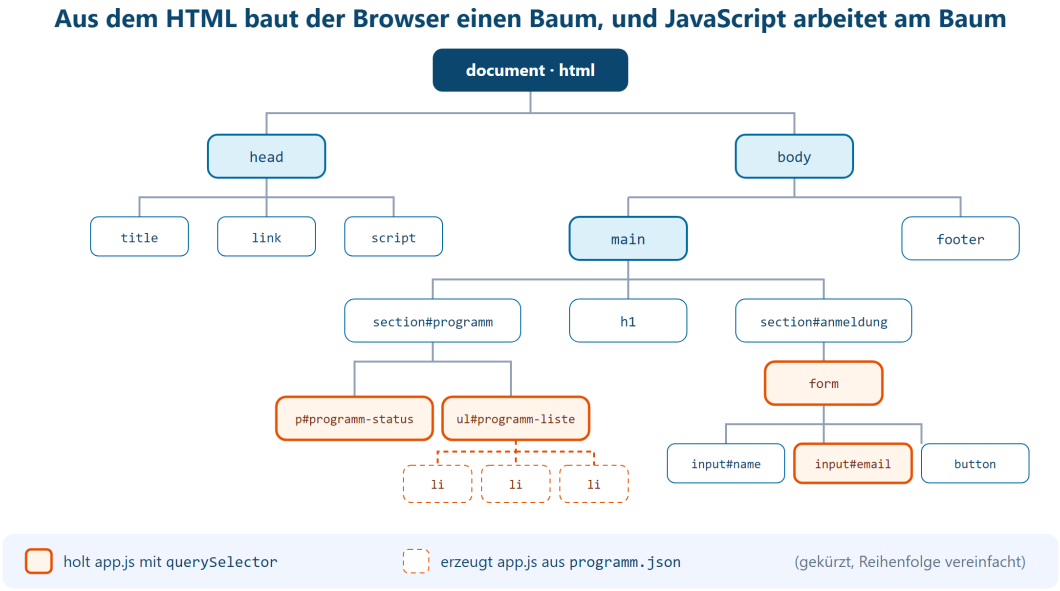


Abbildung 1: Der DOM-Baum der Hackathon-Seite. Orange: Knoten, die `app.js` holt. Gestrichelt: Knoten, die `app.js` erst erzeugt.

Elemente finden und verändern

Mit `document.querySelector` holt ihr ein Element über einen CSS-Selektor, wie ihr ihn aus VL 2 kennt. `querySelectorAll` liefert alle passenden Elemente.

```
const liste = document.querySelector("#programm-liste");
const status = document.querySelector("#programm-status");

status.textContent = "Programm wird geladen …";
status.classList.add("fehler");
status.remove();
```

`textContent` setzt den Text eines Elements. `classList` fügt CSS-Klassen hinzu oder entfernt sie. Das Aussehen sollte über Klassen und CSS gesteuert werden, nicht über einzelne `style`-Eigenschaften im JavaScript. So bleiben Gestaltung und Verhalten getrennt.

Neue Elemente erzeugen

Für jeden Programmpunkt soll eine Karte wie in VL 2 entstehen. Dazu baut man die Elemente einzeln und hängt sie zusammen:

```
function karte({ zeit, titel, text }) {
  const article = document.createElement("article");
  article.className = "card";
  const h3 = document.createElement("h3");
  h3.textContent = `${zeit} · ${titel}`;
  const p = document.createElement("p");
  p.textContent = text;
  article.append(h3, p);
  return article;
}
```

Kürzer wäre es, einen HTML-String zu bauen und über `innerHTML` einzusetzen. Das ist gefährlich, sobald die Daten nicht von euch stammen. Steht in einem Titel etwa `<img src=x onerror="…">`, wertet der Browser das als HTML aus und führt fremden Code aus. Dieser Angriff heißt **Cross-Site Scripting** (XSS) und ist Thema von VL 12.

> ★ Daten setzt ihr mit `textContent` ein, nie mit `innerHTML`. Mit `textContent` bleibt jedes Zeichen Text, auch spitze Klammern.

Events

Webseiten reagieren auf Ereignisse: Klicks, Tastendrücke, das Absenden eines Formulars. Mit `addEventListener` verbindet ihr ein Ereignis mit einer Funktion:

```
const email = document.querySelector("#email");

email.addEventListener("input", () => {
```

```
  const ok = email.value.trim().endsWith("@htwg-konstanz.de");
  email.classList.toggle("ungueltig", !ok);
});
```

Die Funktion wird übergeben, nicht aufgerufen. Sie läuft erst, wenn das Ereignis eintritt, hier bei jedem Tastendruck im Feld. Häufig sind `click`, `input` und `submit`. Wichtig: `submit` gehört auf das `form`, nicht auf den Button. Nur so fangt ihr auch das Absenden per Enter-Taste ab.

Das Standardverhalten abschalten

Klickt jemand auf „Anmelden“, schickt der Browser das Formular ab und lädt eine neue Seite. Mit `event.preventDefault()` verhindert ihr das und übernehmt selbst:

```
form.addEventListener("submit", (event) => {
  event.preventDefault();
  if (!pruefeEmail()) return;
  const danke = document.createElement("p");
  danke.textContent = `Danke, ${form.elements.name.value}!`;
  form.replaceWith(danke);
});
```

Genau so arbeiten Single-Page-Apps: Die Seite wird nie neu geladen, JavaScript tauscht nur Teile des DOM aus und schickt Daten im Hintergrund.

> ⚠ **Achtung:** Eine Prüfung im Browser ist Komfort, kein Schutz. Wer will, umgeht sie mit den Entwicklerwerkzeugen. Der Server muss alle Eingaben noch einmal prüfen. Das behandeln wir in VL 8 und VL 12.

Asynchron: Event Loop und Fetch

Ein Thread und der Event Loop

JavaScript im Browser läuft auf **einem einzigen Thread**. Trotzdem lädt eine Seite Daten nach, während ihr scrollt und klickt. Möglich macht das der **Event Loop**. Langsame Dinge wie Timer oder Netzwerkanfragen erledigt der Browser selbst, außerhalb von JavaScript. Ist er fertig, legt er die zugehörige Funktion in eine Warteschlange. Der Event Loop holt sie dort ab, sobald gerade kein anderer Code läuft.

Dabei gibt es zwei Warteschlangen. Callbacks von Promises landen in der **Microtask Queue**, Timer und Ereignisse wie Klicks in der **Task Queue**. Der Ablauf ist immer derselbe: Erst läuft der aktuelle Code vollständig durch, dann alle Microtasks, dann der nächste Task. Ein Rätsel macht das sichtbar:

```
console.log("Start");
setTimeout(() => console.log("Timer"), 0);
Promise.resolve().then(() => console.log("Promise"));
console.log("Ende");
```

Die Ausgabe lautet `Start`, `Ende`, `Promise`, `Timer`. Der Timer mit 0 Millisekunden kommt zuletzt, weil er erst als Task drankommt, nachdem der aktuelle Code und alle Microtasks erledigt sind.

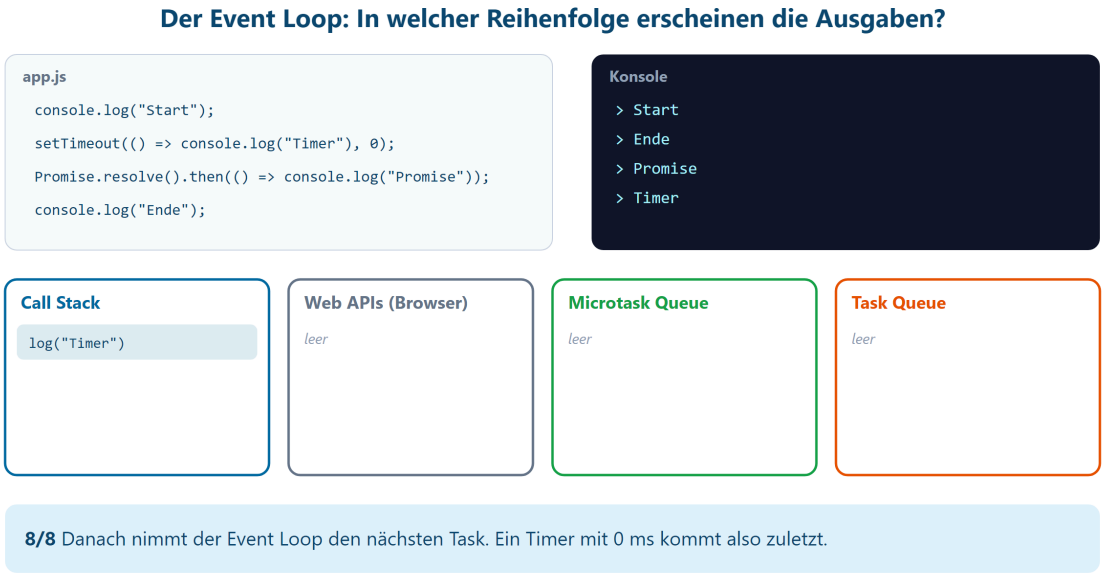


Abbildung 2: Der Endzustand des Rätsels. In der Vorlesung läuft der Ablauf als Animation in acht Schritten.

Daraus folgt eine wichtige Regel: Rechnet synchroner Code lange, friert die ganze Seite ein. Keine Klicks, kein Scrollen, keine Animation, denn der Event Loop kommt nicht dran.

Promises, async und await

Ein **Promise** ist ein Versprechen auf einen Wert, der später kommt, zum Beispiel die Antwort eines Servers. Mit `async` und `await` schreibt ihr Code mit Promises so, als liefe er der Reihe nach:

```
async function ladeProgramm() {
  try {
    const antwort = await fetch("programm.json");
    if (!antwort.ok) throw new Error(`HTTP ${antwort.status}`);
    const punkte = await antwort.json();
    punkte.forEach((p) => liste.append(karte(p)));
    status.remove();
  } catch (fehler) {
    status.textContent = `Fehler: ${fehler.message}`;
  }
}
```

An jedem `await` gibt die Funktion den Thread frei, bis das Ergebnis da ist. Der Rest der Seite bleibt in dieser Zeit bedienbar. Fehler fangt ihr wie in Java mit `try` und `catch`.

Fetch und HTTP

`fetch` schickt eine HTTP-Anfrage, genau die Anfragen aus VL 3, die ihr im Network-Tab seht. Ohne weitere Angaben ist es ein `GET`. Die Antwort enthält Statuscode, Header und Body. Mit `antwort.json()` lest ihr den Body als JSON.

> ⚠ **Achtung:** `fetch` wirft nur bei Netzwerkfehlern eine Ausnahme, also wenn der Server gar nicht erreichbar ist. Ein `404` oder `500` ist für `fetch` eine ganz normale Antwort. Prüft deshalb immer selbst `antwort.ok`, das ist bei allen Statuscodes von 200 bis 299 wahr.

Ausblick: Module und TypeScript

Mit wachsendem Code verteilt man ihn auf mehrere Dateien. Ein **Modul** gibt mit `export` frei, was andere nutzen dürfen, und holt sich mit `import`, was es braucht:

```
// karte.js
export function karte(punkt) { /* … */ }

// app.js
import { karte } from "./karte.js";
```

TypeScript ist JavaScript mit Typangaben. Ein Werkzeug prüft sie, bevor der Code läuft, und übersetzt danach in normales JavaScript:

```
type Programmpunkt = { zeit: string; titel: string; text: string };

function karte(p: Programmpunkt): HTMLElement { /* … */ }
```

Tippfehler wie `p.titl` oder ein vergessenes Feld fallen damit sofort im Editor auf, nicht erst beim Ausprobieren. Nächste Woche legen wir mit **Vite** ein React-Projekt mit TypeScript an. Alles aus diesem Kapitel bleibt dabei gültig: React erzeugt DOM-Elemente, reagiert auf Events und lädt Daten mit `fetch`.

Typische Fehler

⚠ **Achtung:**

- **Seite per Doppelklick geöffnet:** Modul und `fetch` werden blockiert. Lösung: lokaler Server.
- `await` **vergessen:** Statt der Daten steht ein `Promise` in der Variablen, und `forEach` meldet, es sei keine Funktion.
- `antwort.ok` **nicht geprüft:** Bei einem `404` versucht der Code, eine HTML-Fehlerseite als JSON zu lesen, und scheitert mit einer verwirrenden Meldung.
- `submit` **auf dem Button statt auf dem Formular:** Absenden per Enter wird nicht abgefangen, die Seite lädt neu.
- `innerHTML` **mit fremden Daten:** öffnet die Tür für XSS.
- `==` **statt** `===`**:** unerwartete Umwandlungen, etwa `"0" == false`.

Zusammenfassung

★ Ein Skript bindet ihr mit `<script type="module">` ein und liefert die Seite über einen lokalen Server aus. JavaScript ist dynamisch typisiert. Ihr nutzt `const` und `let`, vergleicht mit `===` und behandelt Funktionen als Werte. Der Browser stellt die Seite als DOM-Baum bereit: `querySelector` findet Elemente, `textContent` und `classList` verändern sie, `createElement` und `append` erzeugen neue. Mit `addEventListener` reagiert ihr auf Ereignisse, `preventDefault` schaltet das Standardverhalten ab. JavaScript hat einen Thread, der Event Loop arbeitet erst den aktuellen Code, dann die Microtasks und dann den nächsten Task ab. Daten lädt ihr mit `fetch`, `async` und `await` und prüft dabei immer den Statuscode.

Kontrollfragen

1. Eine Kommilitonin öffnet ihre Seite per Doppelklick. Die Liste bleibt leer, in der Konsole steht ein CORS-Fehler. Was ist die Ursache, und wie behebt sie das Problem?
2. Warum ist `liste.innerHTML += \`
${titel}
\`` gefährlicher als `li.textContent = titel`? Nennt ein konkretes Beispiel für einen Titel, der Schaden anrichtet.
3. Ein Skript enthält eine Schleife, die drei Sekunden rechnet. Warum reagiert die Seite in dieser Zeit nicht auf Klicks, obwohl der Button einen Event-Listener hat?

4. `fetch("programm.json")` liefert einen `404`. Welcher Zweig von `try` und `catch` wird ausgeführt, wenn ihr `antwort.ok` nicht prüft, und was passiert dann beim Aufruf von `antwort.json()`?
5. Gebt die Reihenfolge der Ausgaben an und begründet sie:
   ```
   setTimeout(() => console.log("A"), 0); console.log("B"); Promise.resolve().then(() => console.log("C"));
   ```

Quellen

- Marijn Haverbeke: *Eloquent JavaScript.* 4. Auflage, 2024, Kapitel 1 bis 4, 11, 14, 15 und 18. Frei verfügbar unter eloquentjavascript.net.
- MDN Web Docs: *JavaScript Guide*, *Introduction to the DOM*, *Introduction to events*, *Using the Fetch API*, *Using promises*.
- MDN Web Docs: *The event loop* und *In depth: Microtasks and the JavaScript runtime environment*.
- TypeScript Handbook: *TypeScript for JavaScript Programmers*, typescriptlang.org.